

Entwicklung eines graphischen Editors für die Audiosignalverarbeitung durch VST-Plugins

Johannes Unger

6. Januar 2012

Inhaltsverzeichnis

1	Einleitung	3
1.1	Motivation	3
1.2	Der Name	5
2	Anforderungen	6
2.1	Was implementiert werden muss	6
2.1.1	Die Pluginschnittstelle	6
2.1.2	Der Editor	6
2.1.3	Die Plugindatenbank	7
2.1.4	Programmparameter	8
2.1.5	Parameter Verbindungen	8
2.1.6	Zusätzliche Module	8
2.1.7	Persistenz	9
2.1.8	Latenz Kompensation	9
2.2	Was implementiert werden kann	10
2.2.1	Weitere zusätzliche Module	10
2.2.2	Unterstützung alternativer Pluginformate	10
3	Literatur Verzeichnis	11
4	Abbildungs Verzeichnis	12

1 Einleitung

1.1 Motivation

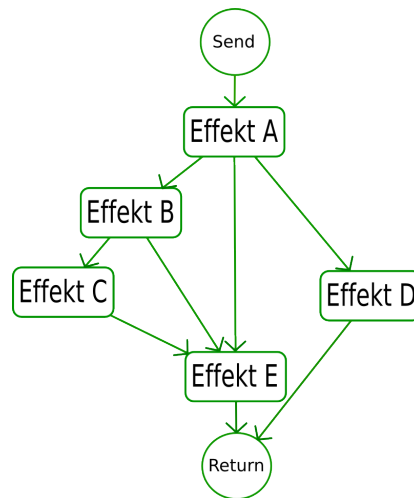


Abbildung 1: Beispiel einer Effektverkettung wie sie in der Realität möglich wäre.

Moderne DAWs¹ bieten umfangreiche und kreative Werkzeuge zum erzeugen und manipulieren von Klangereignissen. In vielen Fällen orientierten sich solche Programme anhand real existierender Hardware. So wird oftmals ein Klangereignis einem Kanal in einem virtuellen Mischpult zugeordnet, der einem real existierenden Gerät nachempfunden wurde.

Es können neben der Lautstärkeregelung und Stereoverteilung auch Effekte in einen solchen Kanal eingeschliffen werden. Hier unterscheiden sich allerdings die Möglichkeiten zwischen real existierender Hardware und der Software Lösung. Während es in der Realität unzählige Möglichkeiten gibt verschiedene Effekte zu kombinieren - sprich zu „verdrahten“ (siehe Abbildung 1) - sind die meisten DAWs in ihren Möglichkeiten beschränkt. Dies soll anhand des Sequenzers „Cubase²“ veranschaulicht werden:

Jede Klangquelle in Cubase, sei es eine Audiospur oder ein virtuelles Instrument, wird einem eigenen Kanal in dem virtuellen Mischpult zugeordnet. Jeder Kanal hat neben einem Equalizer und einer Pan-/Lautstärkeregelung eine feste Anzahl von „Insert-Slots“. In einen solchen „Slot“ kann genau ein Effektmodul geladen werden (in Abbildung 2 und 3 als „Effekt A..X“ bezeichnet). Jeder dieser Effekte wird nacheinander durchlaufen, was einer Reihenschaltung entspricht. Für eine Parallelschaltung steht eine begrenzte Anzahl von sogenannten „Send“ Signalwegen zur Verfügung, von denen es zwei Varianten gibt:

¹Digital Audio Workstation: computergestütztes System für Tonaufnahme, Musikproduktion, Abmischung und Mastering

²verwendete Version: Cubase SL 2.0 (<http://www.steinberg.net>)

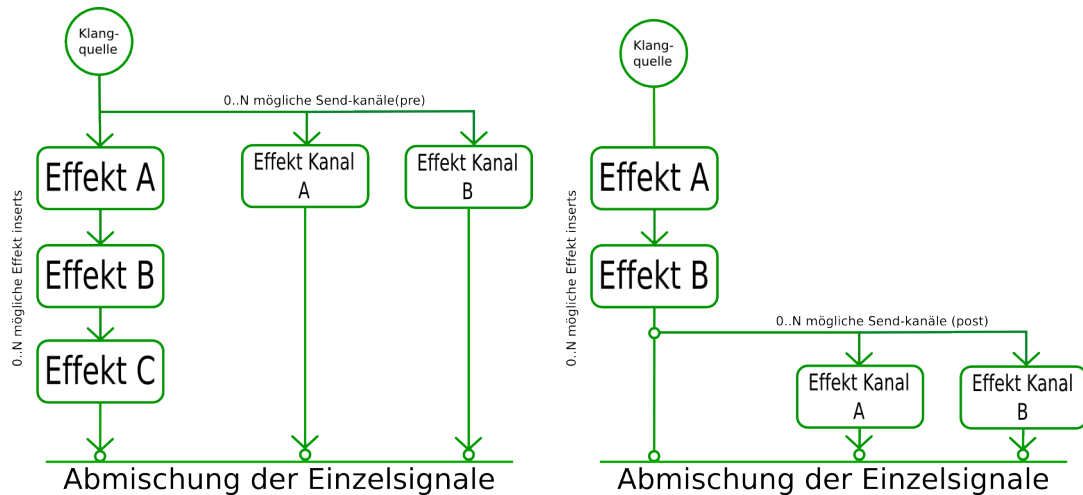


Abbildung 2: Effektkette mit 'send' Abzweigung, jeweils als 'pre' und als 'post' Variante (v.l.n.r.).

"pre-send" und "post-send" (siehe Abbildung 2). Je nach Typ wird das Audiosignal vor der Insert-Effektkette abgegriffen oder danach und in einen Effekt Kanal geleitet. Dieser Effektkanal besteht wiederum aus einer begrenzten Anzahl aus "Insert-Slots".

Diese Möglichkeiten sind für die meisten Produktionen vollkommen ausreichend, da der unterschied ob ein Effekt in Reihe oder Parallel geschaltet wurde - abhängig vom Effekttyp - meistens sehr gering ist. Dennoch ist es manchmal wünschenswert etwas mehr Kontrolle und Übersicht über die Zusammensetzung der einzelnen Signalwege zu haben.

Ziel ist es ein Programm zu Entwickeln das genau das bietet. Es sollte es möglich machen beliebig viele Effekte auf eine virtuelle "Bühne" zu laden um diese wiederum beliebig "verdrahten" zu können. Das Programm sollte selbst ein Effekt sein. Somit wäre die Integration in eine DAW Umgebung denkbar einfach (siehe Abbildung 3).

Die meisten Effekte werden durch Parameter gesteuert. Diese Parameter sollten sich ebenfalls in Form von Reglern auf die "Bühne" holen und von dort aus bedienen lassen. Diese Regler könnte man verbinden und somit mehrere Regler mit einmal steuern. So wäre es zum Beispiel möglich die Intensität eines Halleffektes und die Intensität eines Verzerrers mit einem einzigen Regler gleichzeitig zu steuern. Darüber hinaus bietet es sich an den Signalfluss und das Verhalten der Effekte durch zusätzliche Module manipulierbar beziehungsweise erweiterbar zu machen. Es wäre zum Beispiel eine Art "Step-Sequencer" denkbar. Ein Modul welches Rhythmisch zwischen verschiedenen Signalwegen umschaltet.

Das Programm wäre ein ideales Werkzeug zum erschaffen von kreativen Klangexperimenten.

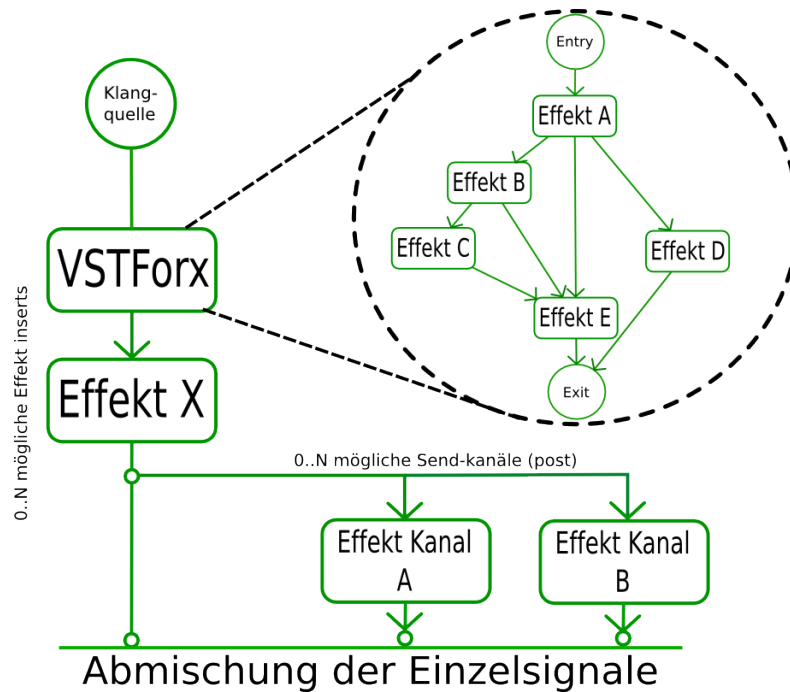


Abbildung 3: Die Idee - freie Verdrahtung als Effekt.

1.2 Der Name

Als Programmname entschied ich mich für "VSTForx".

VST VST³ ist eine Plugin-Schnittstelle im Bereich Audioverarbeitung. Aufgrund der hohen Verbreitung von VST-Plugins, wird die zu entwickelnde Software diese Schnittstelle hauptsächlich unterstützen.

Forx ein Kunstwort das vom englischen Wort "fork" abgeleitet ist. Fork bedeutet Gabelung oder Abzweigung und beschreibt somit ziemlich treffend den Hauptzweck der Software.

³Abkürzung für "Virtual Studio Technology" - Steinberg Media Technologies GmbH

2 Anforderungen

2.1 Was implementiert werden muss

2.1.1 Die Pluginschnittstelle

Da VSTForx ein VST-Plugin werden soll muss es dem VST Standard entsprechen. Es stehen hierbei zwei Versionen zur Auswahl: VST2.x und VST3.x.

VST2.x Das VST2.x SDK hatte im Jahre 2006 sein letztes Update erfahren[Stein06]. Seine Ursprünge reichen bis in das Jahr 1990 zurück als die VST Schnittstelle erstmals entwickelt wurde. Da in den 90er Jahren die Rechenleistung begrenzt war, war ein wesentliches Entwurfsziel die Performance. So ist ein Großteil des SDKs in C geschrieben⁴. Aus heutiger Sicht können viele in der VST2.x SDK verwendete Techniken als veraltet betrachtet werden. So wird zum Beispiel zur 'Host nach Plugin Kommunikation' lediglich eine einzige Methode verwendet die über sogenannte Opcodes⁵ parametrisiert werden. Es werden sehr häufig '*void' Zeiger benutzt was zusammen mit einer schlechten Dokumentation schnell zu schwer auffindbaren Fehlern führen kann.

VST3.x Das VST3.x SDK wurde im Jahre 2008 eingeführt und ist eine vollständige Neuentwicklung[Stein08]. Es ist komplett Objektorientiert und bietet eine Menge neuer Funktionen die dabei helfen VST-Plugins schneller und komfortabler zu entwickeln.

Obwohl aus rein technischer Sicht die Entwicklung eines VST3.x-Plugins lukrativer wäre, fiel die Wahl dennoch auf die VST2.x Version. VST2.x-Plugins sind weiter verbreitet und werden von fast allen DAWs unterstützt.

Ein VST-Plugin tritt in der Form einer Dynamisch ladbaren Bibliothek auf. Je nach System ist es unter Windows eine ".dll" Datei oder unter Mac OSX ein sogenanntes "Resource Bundle".

2.1.2 Der Editor

Der Editor ist die Graphische Schnittstelle zu VSTForx. Er sollte folgende Dinge beinhalten:

Die Bühne Die Bühne ist die Hauptaktionsebene und sollte somit den größten Teil des Editorfensters ausmachen. Die Effektverkettung wird hier erstellt, dargestellt und kann von hier aus auch verändert werden. Plugin Parameter lassen sich als Regler platzieren und bedienen. Ausserdem sollte es von der Bühne aus möglich sein die Editoren der geladenen Plugins zu öffnen und zu benutzen.

⁴es existieren aber - um die Implementierung zu vereinfachen - C++ Wrapperklassen.

⁵Abkürzung für Operation Code. Jede aufrufbare Operation wird durch einen Code Symbolisiert und als Parameter übergeben.

Die Toolbox Um die Bühnenfunktionalität umzusetzen bedarf es einer Unterscheidung von verschiedenen Mausektionen. So ist es ein Unterschied ob man zum Beispiel einen Regler dreht oder ihn verschiebt. Die Mausektion ist aber in beiden Fällen das "ziehen"⁶. Es werden also verschiedene Mausekontexte benötigt. Diese lassen sich in der Toolbox in Form von Knöpfen auswählen.

Das Menü Über ein Menü sollten folgende Funktionen erreichbar sein:

- hinzufügen und entfernen von Bühnenobjekten
- manipulation bzw. anpassen von Bühnenobjekten

Da die meisten Menüfunktionen abhängig vom jeweiligen Bühnenobjekt sind, ist es empfehlenswert das Menü in Form eines Kontextmenüs zu implementieren. Ausserdem sollte es zur Übersichtlichkeit möglich sein das Menü in Unterpunkte aufzuteilen.

Der Setupdialog Von hier aus lassen sich VST-Pluginverzeichnisse festlegen und der "Scanvorgang" (siehe 2.1.3) kann von hier aus gestartet werden. Es sollte von hier aus auch möglich sein die Editorfenstergröße zu bestimmen.

2.1.3 Die Plugindatenbank

Um ein Plugin öffnen zu können würde es im einfachsten Fall ausreichen das Plugin mittels eines "Datei öffnen" Dialoges zu lokalisieren. Dies würde aber schnell umständlich da mit jedem zu öffnenden Plugin durch eine Ordnerstruktur navigiert werden müsste, die zum Großteil aus (Plugin-) uninteressanten Informationen besteht. Ausserdem kann - zumindest unter Windows - nicht zwischen einer System ".dll"Datei und einer VST-Plugin Datei unterschieden werden. Hier bestünde die "Gefahr" eine Datei öffnen zu wollen die gar kein VST-Plugin ist.

Ein weiteres Problem ist es dass VSTForx in der Lage sein soll den kompletten Bühneninhalt zu speichern und zu einem späteren Zeitpunkt wieder zu laden. Würde nun in solch einer Speicherdatei auf absolute Pfade verwiesen, bestünde die Gefahr dass zu einem späteren Zeitpunkt sich die Ordnerstruktur⁷ geändert hat und der gesicherte Pfad somit ungültig wäre.

Sicherer wäre es eine Datenbank zu haben die alle ladbaren Plugins enthält und zur Verfügung stellt. Durch einen einmaligen "Scanvorgang" würde die bestehende Ordnerstruktur durchlaufen und alle gefundenen Plugindateien würden der Datenbank hinzugefügt werden. Sollte sich einmal die Ordnerstruktur ändern müsste nur der "Scanvorgang" wiederholt werden. Für Speicherdateien wäre das Plugin weiterhin verfügbar.

⁶engl.: Drag

⁷oder die Speicherdatei wird auf einem anderen Gerät geöffnet

2.1.4 Programmparameter

Alle Objekte die in VSTForx durch Parameter steuerbar sind sollten ihre Parameter als Regler für die "Bühne" verfügbar machen. Diese Regler sind verbindbar.

Ausserdem sollte das Programm eine Menge an (VST-)Parametern bieten. Diese lassen sich ebenfalls auf der "Bühne" als Regler ablegen. Somit ist es möglich alles "regelbare" in VSTForx von der DAW aus zu steuern.

2.1.5 Parameter Verbindungen

Um noch mehr Möglichkeiten zu haben wäre es sinnvoll in eine Parameterverbindungen Operatoren, die das Verhalten einer Verbindung beeinflussen, einfügen zu können. Wäre zum Beispiel Regler A mit Regler B verbunden, würde bei einem "Inverse" Operator, Regler B zudrehen wenn Regler A aufgedreht würde.

Mögliche Operatoren wären:

- Inverse
- Exponential/Logarithmisch ⁸
- Logarithmisch/Exponential
- Offset (ein fester Verschiebungswert)

2.1.6 Zusätzliche Module

Zusätzliche Module könnten als "interne Effekte" bezeichnet werden. Sie können wie Plugins in den Signalweg eingebunden werden und haben die Möglichkeit diesen zu manipulieren. Genau wie Plugins besitzen Module eine Menge an Parametern durch die Modulspezifische Einstellungen möglich sind.

Mögliche Module:

- VolumeNode - ermöglicht die Lautstärkeregelung vom Signalweg
- PanNode - ermöglicht die Stereoverteilung vom Signalweg
- Schalter - ermöglichen das umschalten beliebig vieler Eingänge/Ausgänge.
- Stepsequencer - ermöglichen das automatische rhythmische Umschalten beliebig vieler Eingänge/Ausgänge.

⁸eine Operation hat zwei Richtungen ("A zu B" und "B zu A") wobei "B zu A" die Inverse Operation von "A zu B" sein muss.

2.1.7 Persistenz

Das Programm sollte in der Lage alle Einstellungen und Konstruktionen dauerhaft zu speichern. Dabei ist zu beachten dass - wird VSTForx in einem DAW Projekt benutzt - VSTForx Bestandteil dieses Projektes ist. Wird also ein solches Projekt gespeichert, sollten mit dem erneuten öffnen des Projektes, alle vorherigen VSTForx Konstruktionen vollständig wiederhergestellt werden.

2.1.8 Latenz Kompensation

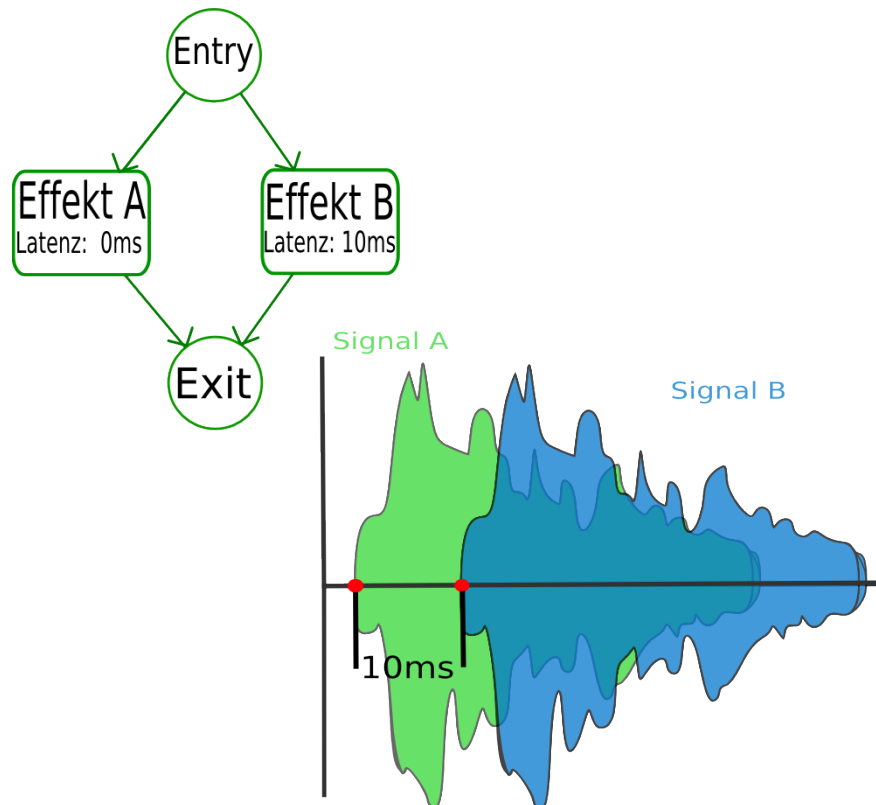


Abbildung 4: Darstellung des Latenzproblems: die Signalausgabe aus "Effekt B" ist um 10 ms verzögert.

Werden in VSTForx zwei Plugins parallel verbunden kann es unter Umständen zu einem Problem kommen: es gibt Plugins die eine Latenzzeit größer 0 aufweisen. Das heißt die Pluginausgabe ist - technisch bedingt - zeitlich verzögert. Bei einer Parallelschaltung kann diese Latenz nun bewirken dass zum Beispiel in einem Signalpfad eine Verzögerung auftritt, im anderen aber nicht (siehe Abbildung 4). Das Signal erklingt daher doppelt. Diese Verzögerung gilt es auszugleichen.

2.2 Was implementiert werden kann

2.2.1 Weitere zusätzliche Module

- Peak2Value - transformiert den Ausschlag (engl. Peak) eines Signals in ein Parameterwert. Solch ein Parameterwert wird als (nicht veränderbarer) Regler dargestellt und kann mit anderen Reglern verbunden werden.
- ADSR2Value - wird an einem Signalpfad ein Schwellwert überschritten, startet dieses Modul eine ADSR⁹ Sequenz. Wie schon beim "Peak2Value" Modul wird der Wert dieser Sequenz in einen Parameterwert transformiert.
- Midi2Value - transformiert MIDI-ereignisse wie Modulation oder Pitch in einen Parameterwert.

2.2.2 Unterstützung alternativer Pluginformate

Denkbar wäre es weitere bekannte (Audio-) Pluginformate zu unterstützen. Sowohl in der Form dass VSTForx alternative Pluginformate kennt und öffnen kann, als auch dass VSTForx in einem alternativen Pluginformat in compilierbar ist.

Alternative Pluginformate wären:

- VST3.x
- AudioUnit (Mac OSX exclusives Pluginformat)
- DirectX Plugin
- LV2 (ist eine OpenSource Audio-Plugin Variante und Weiterentwicklung von LAD-SPA¹⁰)

⁹ADSR - Attack, Decay, Sustain, Release: idealisierten Phasen der Hüllkurve eines Klanges

¹⁰Abkürzung für: Linux Audio Developers Simple Plugin API

3 Literatur Verzeichnis

Literatur

[Stein06] Steinberg Media Technologies GmbH. *Steinberg releases VST 2.4 standard with new features*. <http://www.steinberg.net/index.php?id=334&L=1>, 2006

[Stein08] Steinberg Media Technologies GmbH. *Steinberg releases VST3 SDK*. http://www.steinberg.net/en/company/press/archive/2008/steinberg_vst3_sdk.html, 2008

4 Abbildungs Verzeichnis

Abbildungsverzeichnis

1	Beispiel einer Effektverkettung wie sie in der Realität möglich wäre.	3
2	Effektkette mit 'send' Abzweigung, jeweils als 'pre' und als 'post' Variante (v.l.n.r).	4
3	Die Idee - freie Verdrahtung als Effekt.	5
4	Darstellung des Latenzproblem: die Signalausgabe aus "Effekt B" ist um 10 ms verzögert.	9